



MySQL

SGBD : SYSTÈME DE GESTION DE BASES DE DONNÉES



SOMMAIRE

Introduction
Lire des données avec SELECT
Les jointures
Manipuler des données avec INSERT, UPDATE et DELETE
Altérer une base de données
Les fonctions
Les procédures stockées
Les transactions
Base de données de recettes
Les vues
Les déclencheurs [Trigger]
Les curseurs [Index]
Optimiser les requêtes
La commande SHOW
Importer de données - Assistant WorkBench
Sauvegarde et restauration
Aller plus loin



INTRODUCTION

Quelle est la différence entre SQL et MySQL ?

SQL

SQL pour Structured Query Language est langage permettant d'exploiter une base de données relationnelle. Il comprend de nombreuses commandes telles que SELECT, UPDATE, CREATE, DROP et bien plus.

En fonction des SGBD, le langage SQL comprend quelques différences subtiles de langage.

MySQL

MySQL est un Système de Gestion de Base de Données Relationnelles (SGBDR). Il utilise le langage SQL pour gérer les bases de données relationnelles.

MySQL est largement utilisé pour sa performance, sa simplicité, et sa compatibilité avec des projets de toutes tailles.



LES FONCTIONS

TRAVAUX PRATIQUES

CRÉER DES FONCTIONS SUR LA BASE DE DONNÉES MY_BLOG

La fonction `fn_getFirstNameFromEmail`

Créer une fonction qui permet à partir de l'adresse email d'un utilisateur de retourner le prénom.

La fonction `fn_generatePassword`

Créer une fonction qui génère un mot de passe de 6 caractères.

<https://stackoverflow.com/questions/16737910/generating-a-random-unique-8-character-string-using-mysql>



fn_getFirstNameFromEmail

CORRECTION

Toutes les adresses mails des utilisateurs de la base de données ont le même format, il est donc possible de récupérer le prénom à partir de l'adresse mail.

MySQL n'offre pas de fonction "split", c'est pourquoi il nous faut la créer par nous-même et ensuite utiliser celle-ci pour créer la fonction ci-contre "fn_getEmailFromFirstName".

```
CREATE FUNCTION `fn_split` (p_value VARCHAR(255), p_delimiter VARCHAR(12), p_position INT)
RETURNS VARCHAR(255)
BEGIN
RETURN REPLACE(SUBSTRING(SUBSTRING_INDEX(p_value, p_delimiter, p_position),
                        LENGTH(SUBSTRING_INDEX(p_value, p_delimiter, p_position - 1)) + 1),
                p_delimiter, '');
END
```

fn_split

```
CREATE FUNCTION `fn_getFirstNameFromEmail` (p_email VARCHAR(150))
RETURNS VARCHAR(80)
BEGIN
RETURN fn_split(fn_split(p_email, '@', 1), '.', 1);
END
```

fn_getEmailFromFirstName



fn_generatePassword

CORRECTION

La fonction "fn_generatePassword" prend en paramètre un nombre entier définissant la taille (nombre de caractères) du mot de passe à générer.

```
CREATE FUNCTION `fn_generatePassword` (p_size INT)
RETURNS VARCHAR(40)
BEGIN
    DECLARE result VARCHAR(40) DEFAULT '';
    DECLARE allowedChars VARCHAR(52) DEFAULT 'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ...';

    WHILE p_size > 0 DO
        SET result = CONCAT(result,
                            substring(allowedChars, FLOOR(RAND() * LENGTH(allowedChars) + 1), 1));
        SET p_size = p_size - 1;
    END WHILE;

    RETURN result;
END
```

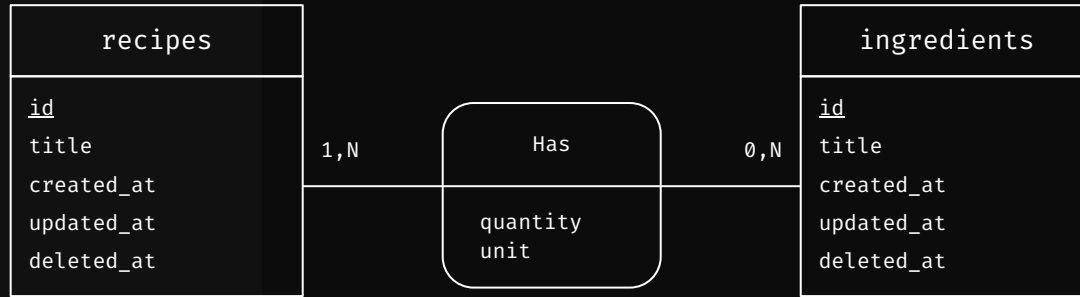




LA BASE DE DONNÉES : MY_RECIPES

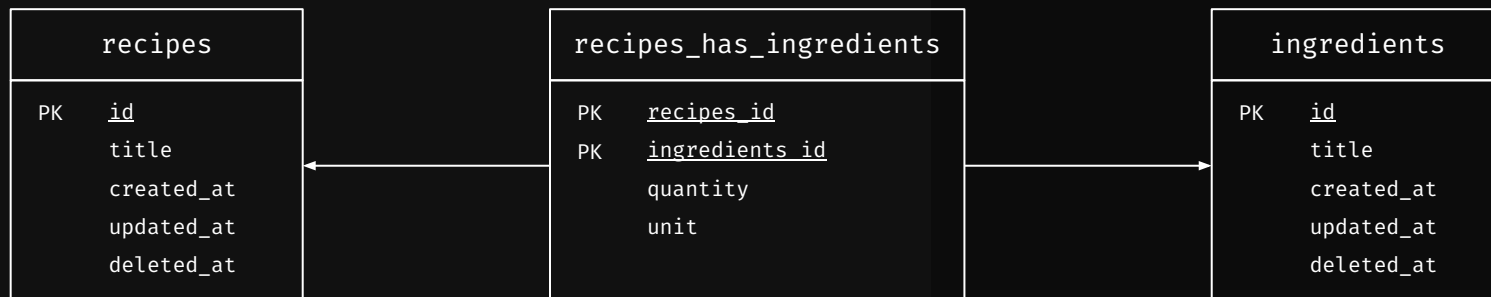
MODÈLE CONCEPTUEL DE DONNÉES

LA BASE DE DONNÉES MY_RECIPES



MODÈLE PHYSIQUE DE DONNÉES

LA BASE DE DONNÉES MY_RECIPES



SCRIPT SQL

LA BASE DE DONNÉES MY_RECIPES

À partir du modèle physique de données créé sur MySQL Workbench, il est possible de générer le script SQL à exécuter afin d'obtenir la base de données.

Vous avez ci-contre un extrait du script SQL. La version entière du script à vous sera fourni au moment opportun.

```
--  
-- Schema my_recipes  
--  
CREATE SCHEMA IF NOT EXISTS `my_recipes` DEFAULT CHARACTER SET utf8 ;  
USE `my_recipes` ;  
  
--  
-- Table `my_recipes`.`recipes`  
--  
CREATE TABLE IF NOT EXISTS `my_recipes`.`recipes` (  
  `id` INT NOT NULL AUTO_INCREMENT,  
  `title` VARCHAR(80) NOT NULL,  
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  `deleted_at` TIMESTAMP NULL DEFAULT NULL,  
  PRIMARY KEY (`id`),  
  UNIQUE INDEX `title_UNIQUE` (`title` ASC) VISIBLE)  
ENGINE = InnoDB;  
  
--  
-- Table `my_recipes`.`ingredients`  
--  
CREATE TABLE IF NOT EXISTS `my_recipes`.`ingredients` (  
  `id` INT NOT NULL AUTO_INCREMENT,  
  `title` VARCHAR(80) NOT NULL,  
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  `deleted_at` TIMESTAMP NULL DEFAULT NULL,  
  PRIMARY KEY (`id`),  
  UNIQUE INDEX `title_UNIQUE` (`title` ASC) VISIBLE)  
ENGINE = InnoDB;  
  
[ ... ]
```

my_recipes.sql



LA BASE DE DONNÉES MY_RECIPES

JEU DE DONNÉES

LA BASE DE DONNÉES MY_RECIPES

Une fois la structure de la base de données obtenue, il est nécessaire d'exécuter un second script SQL pour avoir un jeu de données exploitable.

Vous avez ci-contre un extrait du script SQL. La version entière du script à vous sera fourni au moment opportun.

```
-- RECIPES: COUNT 3
INSERT INTO recipes (`title`)
VALUES
    ('Gâteau au chocolat'),
    ('Bricks au fromage et viande hachée'),
    ('Praliné maison');

-- INGREDIENTS: COUNT 18
INSERT INTO ingredients (`title`)
VALUES
    ('Noix'),
    ('Crème fraîche'),
    ('Farine'),
    ('Chocolat noir'),
    ('Beurre'),
    ('Poudre de noisette'),
    ('Oeufs'),
    ('Fleur de sel'),
    ('Sucre en poudre'),
    ('Amandes'),
    ('Noisettes'),
    ('Huile de tournesol'),
    ('Poivre'),
    ('Sel'),
    ('Oignons'),
    ('Persil'),
    ('Viande hachée'),
    ('Feuilles de brick');

-- RECIPES_HAS_INGREDIENTS (PIVOT TABLE)
INSERT INTO recipes_has_ingredients (`recipes_id`, `ingredients_id`, quantity, unit)
VALUES
    (1, 1, 15, NULL),
    (1, 2, 2, 'Cuillère à soupe'),
    (1, 3, 50, 'Gramme'),
    (1, 9, 120, 'Gramme'),
    (1, 4, 160, 'Gramme'),
    (1, 5, 110, 'Gramme'),
    (1, 6, 60, 'Gramme'),
    (1, 7, 3, NULL),
```

[...]

dataset.sql



LA BASE DE DONNÉES MY_RECIPES



LES VUES

QU'EST-CE QU'UNE VUE ?

DÉFINITION

Une vue est une requête SQL stockée qui est traitée comme une table virtuelle. Une vue est une représentation structurée des données dans une ou plusieurs tables de la base de données, mais ce n'est pas une table réelle.

Les vues sont souvent utilisées pour simplifier les requêtes complexes, réutiliser des requêtes fréquemment utilisées, ou fournir une couche de sécurité en limitant l'accès direct aux tables sous-jacentes.

V

I

E

W

UNE REQUÊTE FRÉQUEMMENT UTILISÉE

EXEMPLE

Vous êtes développeur backend, et vous travaillez pour l'entreprise Marmithon. Vous vous rendez compte que depuis quelques jours, vous utilisez une même requête très souvent. Celle-ci permet de connaître l'ensemble des ingrédients de chacune des recettes.

```
SELECT r.title, GROUP_CONCAT(i.title, '') AS ingredients
FROM recipes r
INNER JOIN recipes_has_ingredients ri ON r.id = ri.recipes_id
INNER JOIN ingredients i ON ri.ingredients_id = i.id
GROUP BY r.title;
```

Requête SQL

title	ingredients
Bricks au fromage et viande hachée	Huile de tournesol ,Poivre ,Sel ,Oignons ,Persil ,Viande hachée ,Feuilles de brick
Gâteau au chocolat	Noix ,Crème fraîche ,Farine ,Chocolat noir ,Beurre ,Poudre de noisette ,Oeufs ,Sucre en poudre
Praliné maison	Fleur de sel ,Sucre en poudre ,Amandes ,Noisettes

Ensemble de résultats

Réécrire cette requête à chaque fois que l'on en a besoin peut vite devenir fastidieux, c'est là que les vues en MySQL prennent tout leur sens.

CRÉATION DE LA VUE

V_RECIPES_WITH_INGREDIENTS

La commande SQL "CREATE VIEW" permet de créer une vue. La requête ci-contre permet de créer la vue "v_recipes_with_ingredients". À l'aide de jointures internes, cette vue permet de connaître facilement les ingrédients de chacune des recettes.

```
CREATE VIEW v_recipes_with_ingredients AS
  SELECT r.title, GROUP_CONCAT(i.title, '') AS ingredients
  FROM recipes r
  INNER JOIN recipes_has_ingredients ri ON r.id = ri.recipes_id
  INNER JOIN ingredients i ON ri.ingredients_id = i.id
  GROUP BY r.title;
```

UTILISATION DE LA VUE

V_RECIPES_WITH_INGREDIENTS

Lorsque la vue est créée, elle s'utilise exactement de la même manière qu'une table. Le nom de la vue se précise après la commande "FROM".

```
SELECT * FROM v_recipes_with_ingredients;
```

Requête SQL

	title	ingredients
►	Bricks au fromage et viande hachée	Huile de tournesol ,Poivre ,Sel ,Oignons ,Persil ,Viande hachée ,Feuilles de brick
	Gâteau au chocolat	Noix ,Crème fraîche ,Farine ,Chocolat noir ,Beurre ,Poudre de noisette ,Oeufs ,Sucre en poudre
	Praliné maison	Fleur de sel ,Sucre en poudre ,Amandes ,Noisettes

Ensemble de résultats



SUPPRESSION D'UNE VUE

Pour supprimer une vue, il suffit d'utiliser la commande "DROP VIEW" en précisant le nom de la vue à supprimer. Pour éviter les erreurs, vous pouvez utiliser la commande "DROP VIEW IF EXISTS".

Dans l'exemple ci-contre, nous supprimons la vue "v_recipes_with_ingredients".

```
DROP VIEW v_recipes_with_ingredients;
```



LES DÉCLENCHEURS [TRIGGER]

QU'EST-CE QU'UN TRIGGER ?

DÉFINITION

Un trigger (ou déclencheur) est une opération SQL qui est automatiquement invoquée lors d'une commande INSERT, DELETE ou UPDATE. Le code d'un trigger est associé à une table et est détruit une fois que la table est supprimée.

Un exemple pourrait être un trigger qui enregistre des informations dans une table "logs" chaque fois qu'un nouvel enregistrement est insérée dans la table en question. Regardons ensemble quels sont les événements à partir desquels un trigger peut être exécuté.

T
R
I
G
G
E
R



LES ÉVÈNEMENTS

IL EN EXISTE PLUSIEURS

BEFORE INSERT

AFTER INSERT

BEFORE UPDATE

AFTER UPDATE

BEFORE DELETE

AFTER DELETE



AJOUT DE LA COLONNE USAGE_COUNT

ALTÉRATION DE LA TABLE INGREDIENTS

Avant de créer un trigger, nous allons créer la colonne "usage_count" dans la table "ingredients". Cette colonne verra sa valeur mise à jour avec le nombre d'usages des ingrédients dans les recettes.

```
ALTER TABLE ingredients  
ADD COLUMN `usage_count` INT DEFAULT 0 AFTER `title`;
```



Mini-exercice

Mettre à jour la valeur de `usage_count` avec le nombre de fois où l'ingrédient est utilisé dans les recettes.

CRÉATION DU TRIGGER

UPDATE_USAGE_COUNT_ON_INGREDIENT_LINKED

Dans l'exemple ci-contre, le trigger incrémente de 1 la valeur la colonne "usage_count" lorsqu'un ingrédient est ajouté à une recette. Le lien se fait à travers l'ajout d'un enregistrement dans la table de jointure "recipes_has_ingredients".

```
DELIMITER //
CREATE TRIGGER update_usage_count_on_ingredients_linked
AFTER INSERT ON recipes_has_ingredients FOR EACH ROW
BEGIN
    UPDATE ingredients
    SET usage_count = usage_count + 1
    WHERE id = NEW.ingredients_id;
END
//
DELIMITER ;
```



NEW ET OLD

MIEUX COMPRENDRE

NEW fait référence aux nouvelles valeurs qui sont insérées ou mises à jour après qu'une commande "INSERT" ou "UPDATE" ait été exécutée.

OLD fait référence aux anciennes valeurs avant qu'elles soient mises à jour ou supprimées après qu'une commande "UPDATE" ou "DELETE" ait été exécutée.

À titre d'exemple, vous avez ci-contre un trigger qui recense toutes les modifications apportées aux titres des recettes dans la table "recipes_logs" en stockant l'ancien et le nouveau "titre".

```
DELIMITER //
CREATE TRIGGER history_on_recipes_title_update
AFTER UPDATE ON recipes
FOR EACH ROW
BEGIN
    IF OLD.title <> NEW.title THEN
        INSERT INTO recipes_logs (old_value, new_value)
        VALUES (OLD.title, NEW.title);
    END IF;
END;
//
DELIMITER ;
```



NEW ET OLD EN FONCTION DES ÉVÈNEMENTS

ACCESSIBILITÉ

ÉVÉNEMENTS	VALEURS ACCESSIBLES
BEFORE INSERT - AFTER INSERT	NEW
BEFORE UPDATE - AFTER UPDATE	NEW ET OLD
BEFORE DELETE - AFTER DELETE	OLD

VÉRIFIER SI LE TRIGGER SE DÉCLENCHÉ

AJOUTONS DU MIEL DANS NOTRE GÂTEAU AU CHOCOLAT

Lors de l'ajout de 2 cuillères à café de miel dans la recette du gâteau au chocolat, le trigger "update_usage_count_on_ingredients_linked" se déclenche et incrémente de 1 la valeur de "usage_count". (cf. ci-contre)

```
INSERT INTO ingredients (title) VALUES ('Miel');  
INSERT INTO recipes_has_ingredients (recipes_id, ingredients_id, quantity, unit)  
VALUES (1, 19, 2, 'Cuillère à café');
```

```
SELECT * FROM ingredients WHERE title = 'Miel';
```

Requête SQL

	id	title	usage_count	created_at	updated_at	deleted_at
►	19	Miel	1	2024-05-12 16:55:38	2024-05-12 17:03:01	NULL

Ensemble de résultats



Mini-exercice

Créer le trigger "update_usage_count_on_ingredients_unlinked" qui se déclenche lors de la suppression d'un ingrédient dans une recette.





LES CURSEURS [INDEX]

QU'EST-CE QU'UN INDEX ?

DÉFINITION

Un index est une structure de données qui améliore les performances des requêtes en permettant un accès plus rapide aux données d'une table.

Un index est créé sur une ou plusieurs colonnes d'une table, et il agit comme un guide pour le moteur de base de données afin de trouver rapidement les lignes correspondantes lorsqu'une requête est exécutée.

I
N
D
E
X

Ce que les développeurs doivent savoir sur les index

Nelson Dionisi

Director of Engineering @Miraki



OPTIMISER LES REQUÊTES

LA COMMANDE EXPLAIN

MIEUX COMPRENDRE LE FONCTIONNEMENT D'UNE REQUÊTE

Pour mieux comprendre comment opère le moteur SQL lors de l'exécution d'une requête, la commande EXPLAIN fournit diverses informations intéressantes.

Se soucier de l'optimisation de ses requêtes fait partie des bonnes pratiques et peut s'avérer indispensable lorsque l'on manipule un grand volume de données.

EXPLAIN

```
(  
    SELECT AVG(v.lc) FROM  
    (  
        SELECT articles.title, COUNT(users_has_articles.articles_id) AS lc  
        FROM users  
        INNER JOIN users_has_articles ON users.id = users_has_articles.users_id  
        INNER JOIN articles ON users_has_articles.articles_id = articles.id  
        GROUP BY articles.title  
    ) AS v  
);
```

Requête SQL

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref
►	1	PRIMARY	<derived2>	NULL	ALL	NULL	NULL	NULL	NULL
	2	DERIVED	users_has_articles	NULL	index	PRIMARY, fk_users_h...	fk_users_has_articles_...	4	NULL
	2	DERIVED	users	NULL	eq_ref	PRIMARY	PRIMARY	4	my
	2	DERIVED	articles	NULL	eq_ref	PRIMARY	PRIMARY	4	my

Ensemble de résultats

LA COMMANDE EXPLAIN

MIEUX COMPRENDRE LE FONCTIONNEMENT D'UNE REQUÊTE

id

identifiant de la requête, plus le nombre est élevé, plus l'étape est exécuté tardivement

select_type

type de la requête (SIMPLE, PRIMARY, SUBQUERY, DERIVED, UNION...)

table

Le nom de table concerné par la requête

possible_keys

Liste des index qui pourraient être utilisés par la requête

key

Indique l'index utilisé pour lire les données

key_len

Longueur de l'index utilisé en octets

ref

Nom de la ou les colonnes utilisées avec l'index pour trouver les enregistrements

rows

Estimation du nombre d'enregistrements lus pour trouver les données demandées

filtered

Pourcentage estimé d'enregistrements filtrées par la condition WHERE

Extra

Informations additionnelles sur l'exécution de la requête

EXPLAIN

```
(  
  SELECT AVG(v.lc) FROM  
  (  
    SELECT articles.title, COUNT(users_has_articles.articles_id) AS lc  
    FROM users  
    INNER JOIN users_has_articles ON users.id = users_has_articles.users_id  
    INNER JOIN articles ON users_has_articles.articles_id = articles.id  
    GROUP BY articles.title  
  ) AS v  
);
```

Requête SQL

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref
►	1	PRIMARY	<derived2>	NULL	ALL	NULL	NULL	NULL	NULL
	2	DERIVED	users_has_articles	NULL	index	PRIMARY, fk_users_h...	fk_users_has_articles_...	4	NULL
	2	DERIVED	users	NULL	eq_ref	PRIMARY	PRIMARY	4	my...
	2	DERIVED	articles	NULL	eq_ref	PRIMARY	PRIMARY	4	my...

Ensemble de résultats



La commande EXPLAIN peut aussi décrire une table, regardons comment

LA COMMANDE EXPLAIN / DESCRIBE

OBTENIR LA DESCRIPTION D'UNE TABLE

```
EXPLAIN articles;
```

```
-- Équivalent
```

```
DESCRIBE articles;
```

La commande EXPLAIN permet aussi de connaître la structure d'une table (ensemble des colonnes et leurs spécificités).

Requête SQL

	Field	Type	Null	Key	Default	Extra
►	id	int	NO	PRI	NULL	auto_increment
	title	varchar(255)	NO		NULL	
	slug	varchar(255)	NO		NULL	
	content	longtext	NO		NULL	
	cover	varchar(100)	NO		NULL	
	created_at	timestamp	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED
	updated_at	timestamp	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED
	deleted_at	timestamp	YES		NULL	
	users_id	int unsigned	NO	MUL	NULL	
	categories_id	int	NO	MUL	NULL	

Ensemble de résultats

La commande EXPLAIN fait exactement la même chose que la commande DESCRIBE
<https://dev.mysql.com/doc/refman/8.4/en/describe.html>

“

The DESCRIBE and EXPLAIN statements are synonyms. In practice, the DESCRIBE keyword is more often used to obtain information about table structure, whereas EXPLAIN is used to obtain a query execution plan (that is, an explanation of how MySQL would execute a query)

”

<https://dev.mysql.com/doc/refman/8.4/en/explain.html>



LA COMMANDE SHOW

LA COMMANDE SHOW

OBTENIR DES INFORMATIONS

La commande SHOW permet d'afficher des informations additionnelles sur différents éléments de la base de données.

-- Affiche toutes les bases de données

SHOW DATABASES;

-- Affiche des infos. sur les tables

SHOW TABLE STATUS;

-- Affiche des infos. sur les procédures

SHOW PROCEDURE STATUS;

-- Affiche le nombre d'erreurs d'une base de données

SHOW COUNT(*) ERRORS;

-- Affiche le nombre d'avertissements d'une base de données

SHOW COUNT(*) WARNINGS;





SAUVEGARDE ET RESTAURATION

ARRIVE BIENTÔT



BONUS ET RECOMMANDATIONS

BYTE BYTE GO



ALLER PLUS LOIN

ARRIVE BIENTÔT



MERCI DE VOTRE ATTENTION